

Learning While Doing: Methodology

Reference documentation for the `learning-while-doing` package

August 3, 2026

Contents

1	Overview	2
2	Simulation model	2
2.1	State variables and daily updates	2
2.2	Cost accounting	4
2.3	The learning target	4
2.4	Common Random Numbers	4
3	Belief model: Gaussian Process	5
3.1	Posterior after n observations	5
3.2	Posterior covariance between two query points	5
3.3	Implementation notes	5
4	Acquisition policies	6
4.1	Random baseline	6
4.2	Interval Estimation (IE / LCB)	6
4.3	Knowledge Gradient: offline correlated, analytic	6
4.4	Knowledge Gradient: offline correlated, Monte Carlo	8
4.5	Knowledge Gradient: offline independent	8
4.6	Online Knowledge Gradient (Ryzhov)	9
4.7	Summary of KG variants	9
5	Session workflow	9
6	Batch benchmarking	10
6.1	Families	10
6.2	Common Random Numbers across policies	10
6.3	Metrics	10
6.4	Ground truth reference	11
6.5	Aggregation	11

7	Configuration parameters	11
8	Software architecture	11

1 Overview

The `learning-while-doing` package is an interactive simulation for studying how one might learn the optimal *cash-buffer ratio* θ for a mutual fund while balancing two competing costs:

- **Opportunity cost** — cash sitting in a low-yield instrument earns less than the market portfolio, so holding *too much* cash drags returns.
- **Shortfall cost** — when redemptions exceed available cash, the fund must liquidate assets under adverse conditions, incurring an incremental friction penalty. Holding *too little* cash raises the probability of these events.

The *true expected cost* function $F(\theta)$ is unknown to the decision-maker. It is defined implicitly by the stochastic dynamics of the fund (Section 2). The decision-maker chooses a sequence of candidate values $\theta_1, \theta_2, \dots$, observes a noisy realization of $F(\theta_i)$ after each choice, and updates their belief about F .

The learning problem has two flavors:

- **Offline:** only the terminal decision matters — after a budget of N measurements, one commits to a single θ and lives with the cost of that choice indefinitely.
- **Online:** measurement costs are also incurred during learning — every trial’s cost counts against the total. This is the classical online learning setting.

The package supports both regimes and provides a suite of policies for choosing the next θ : pure exploration (Random), a family of interval-estimation policies (IE), and five variants of the Knowledge Gradient (KG) — offline analytic, offline Monte Carlo, offline independent, online correlated (Ryzhov), and online independent.

Section 2 defines the simulator. Section 3 covers the Gaussian-process belief model. Section 4 derives the acquisition policies. Section 5 describes the session workflow and Common Random Numbers. Section 6 defines the batch benchmarking framework.

2 Simulation model

A single simulation experiment consists of running a deterministic-once-seeded fund path over H trading days at a chosen θ , then computing the total cost incurred. The default horizon is 26 weeks ($H = 130$ trading days).

2.1 State variables and daily updates

Let AUM_t denote assets under management, Cash_t the cash balance, and $\text{Inv}_t = \text{AUM}_t - \text{Cash}_t$ the invested portfolio balance at the end of day t . The initial state is $\text{AUM}_0 = A_0$, $\text{Cash}_0 = \theta A_0$,

$$\text{Inv}_0 = (1 - \theta)A_0.$$

Each day the following updates occur in order.

Regime transition (optional). A two-state hidden Markov chain governs an additive drift term. Let $R_t \in \{0, 1\}$ be the regime. If regime-switching is enabled, $R_{t+1} \neq R_t$ with probability p_{switch} ; otherwise $R_{t+1} = R_t$. Each regime carries a per-year drift adjustment $\mu_{R_t}^{\text{reg}}$.

Retail net flow. Modeled as a Brownian increment on the fraction of AUM. With $\Delta t = 1/\text{tdy}$ (default $\text{tdy} = 252$), the effective daily drift is

$$\mu_t^{\text{eff}} = \mu_{\text{net}}\Delta t + \mu_{R_t}^{\text{reg}}\Delta t,$$

and the retail flow into cash on day t is

$$F_t^{\text{retail}} = \text{AUM}_t(\mu_t^{\text{eff}} + \sigma_{\text{net}}\sqrt{\Delta t} Z_t), \quad Z_t \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1).$$

The default annual drift is $\mu_{\text{net}} = 0$ and the default annual volatility is $\sigma_{\text{net}} = 0.02$.

Institutional jumps. A Poisson process with per-year rate λ generates institutional in/outflow events. The number of events on day t is $N_t \sim \text{Poisson}(\lambda\Delta t)$. Each event $k = 1, \dots, N_t$ has a signed jump size

$$J_t^{(k)} = D_t^{(k)} \cdot \exp(\mu_{\text{jump}} + \sigma_{\text{jump}}\varepsilon_t^{(k)}) \cdot \text{AUM}_t,$$

where $D_t^{(k)} = +1$ with probability p_{in} and -1 otherwise, and $\varepsilon_t^{(k)} \sim \mathcal{N}(0, 1)$. The total institutional flow is $F_t^{\text{inst}} = \sum_{k=1}^{N_t} J_t^{(k)}$. Default parameters: $\lambda = 12/\text{yr}$, $\mu_{\text{jump}} = -4.5$ (median jump $\approx 1.1\%$ of AUM), $\sigma_{\text{jump}} = 0.5$, $p_{\text{in}} = 0.5$.

Cash update and shortfall recording. The net flow hits the cash balance first:

$$\text{Cash}_{t+1}^- = \text{Cash}_t + F_t^{\text{retail}} + F_t^{\text{inst}}.$$

If the pre-rebalance cash would go negative, the shortfall is recorded before the rebalance:

$$S_t = \max(-\text{Cash}_{t+1}^-, 0).$$

Invested growth. Invested assets grow at the daily market rate $r_{\text{mkt}}\Delta t$:

$$\text{Inv}_{t+1}^- = \text{Inv}_t \cdot (1 + r_{\text{mkt}}\Delta t).$$

End-of-day rebalance. Let $\text{AUM}_{t+1}^- = \text{Cash}_{t+1}^- + \text{Inv}_{t+1}^-$. The target cash level is $\theta \cdot \text{AUM}_{t+1}^-$ and the gap is $g_t = \theta \text{AUM}_{t+1}^- - \text{Cash}_{t+1}^-$. The transfer applied is

$$\tau_t = \max(\rho \cdot g_t, -\text{Inv}_{t+1}^-),$$

where $\rho \in (0, 1]$ is the rebalance speed (default 1, i.e. full-rebalance). Post-transfer:

$$\text{Cash}_{t+1} = \text{Cash}_{t+1}^- + \tau_t, \quad \text{Inv}_{t+1} = \text{Inv}_{t+1}^- - \tau_t.$$

2.2 Cost accounting

The realized total cost over the H -day horizon has two additive components.

Opportunity cost. The drag rate is $r_{\text{drag}} = r_{\text{mkt}} - r_{\text{cash}}$ (default $0.07 - 0.04 = 0.03/\text{yr}$). With default option `opp_cost_on_total_cash = True` we apply the drag to *all* cash held:

$$\text{OppCost} = r_{\text{drag}} \Delta t \sum_{t=1}^H \text{Cash}_t.$$

Shortfall cost. A proportional penalty on the total shortfall, independent of time scale:

$$\text{ShortCost} = r_{\text{borrow}} \sum_{t=1}^H S_t.$$

Here r_{borrow} (default 0.10/yr) is the per-dollar friction cost of forced asset liquidation, not a daily rate.

Total cost. The observation returned by the simulator is

$$y = \text{TotalCost} = \text{OppCost} + \text{ShortCost}.$$

By construction, $y \geq 0$.

2.3 The learning target

The unknown function the learner attempts to estimate is

$$F(\theta) \equiv \mathbb{E}[\text{TotalCost}(\theta)],$$

where the expectation is over the randomness in the sim path (regime path, retail flow, jumps). A single simulation call at θ returns $y = F(\theta) + \varepsilon$, where ε is the realization noise; in this model ε is non-Gaussian and heteroscedastic (jump-frequency and jump-size distributions vary the variance across the domain), but the belief model treats it as $\mathcal{N}(0, \tilde{\sigma}^2)$ for tractability.

2.4 Common Random Numbers

The simulator is a pure function of `(config,  $\theta$ ,  $H$ , session_seed, experiment_index)`. Two calls with the same `(session_seed, experiment_index)` pair but different θ share their random draws (regime path, retail Z_t , jump counts N_t and sizes $J_t^{(k)}$). This is Common Random Numbers (CRN):

it dramatically reduces the variance of pairwise cost comparisons across θ values within a single session.

Independent sub-streams for each experiment index are drawn from `numpy.random.SeedSequence.spawn`, which guarantees non-overlapping BitGenerator states.

3 Belief model: Gaussian Process

The learner maintains a Gaussian-process (GP) posterior over F . The prior is

$$F(\cdot) \sim \mathcal{GP}(m_0, k(\cdot, \cdot)),$$

with constant mean m_0 (default 5000) and squared-exponential (RBF) kernel

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{1}{2} \frac{(x - x')^2}{\ell^2}\right),$$

with signal std σ_f (default 5000, in cost units) and length scale ℓ (default 0.04, in θ units).

3.1 Posterior after n observations

Given observations $(x_i, y_i)_{i=1}^n$ with assumed noise model $y_i = F(x_i) + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \tilde{\sigma}^2)$, define

$$\mathbf{K} = [k(x_i, x_j)]_{i,j=1}^n, \quad \mathbf{k}_*(x) = [k(x, x_i)]_{i=1}^n, \quad \mathbf{r} = \mathbf{y} - m_0 \mathbf{1}.$$

The posterior mean and variance at a query point x are

$$\mu_n(x) = m_0 + \mathbf{k}_*(x)^\top (\mathbf{K} + \tilde{\sigma}^2 \mathbf{I})^{-1} \mathbf{r}, \tag{1}$$

$$\sigma_n^2(x) = k(x, x) - \mathbf{k}_*(x)^\top (\mathbf{K} + \tilde{\sigma}^2 \mathbf{I})^{-1} \mathbf{k}_*(x). \tag{2}$$

Because $k(x, x) = \sigma_f^2$ for the RBF kernel, Eq. (2) simplifies to $\sigma_n^2(x) = \sigma_f^2 - \|\mathbf{L}^{-1} \mathbf{k}_*(x)\|^2$, where $\mathbf{L}$ is the lower-triangular Cholesky factor of $\mathbf{K} + \tilde{\sigma}^2 \mathbf{I}$.

3.2 Posterior covariance between two query points

For a set of query points $\{x_1, \dots, x_m\}$, the full posterior covariance matrix is

$$\text{Cov}_n(x_i, x_j) = k(x_i, x_j) - \mathbf{k}_*(x_i)^\top (\mathbf{K} + \tilde{\sigma}^2 \mathbf{I})^{-1} \mathbf{k}_*(x_j).$$

This matrix appears explicitly in the correlated-KG derivations (Section 4.3).

3.3 Implementation notes

The implementation caches the Cholesky factor $\mathbf{L}$ and the vector $\alpha = (\mathbf{K} + \tilde{\sigma}^2 \mathbf{I})^{-1} \mathbf{r}$ after each observation. A small jitter 10^{-6} is added to the diagonal for numerical stability. Posterior evaluation

at m query points costs $O(nm)$ per call after the $O(n^3)$ Cholesky, which is recomputed whenever a new observation is appended.

The GP treats observations as Gaussian, but the true simulator noise is non-Gaussian and can be heteroscedastic. This mis-specification is pedagogical: it lets students see how a "wrong prior" affects the algorithms' behavior.

4 Acquisition policies

An acquisition policy is a rule π that, given the current belief model and any auxiliary state, proposes the next θ to measure. All policies in the package share the interface `propose(model, rng) -> float` and select from a fixed grid $\mathcal{G} = \{\theta_1, \dots, \theta_M\}$ (default $M = 100$ evenly spaced over $[\theta_{\min}, \theta_{\max}]$).

Sections 4.1–4.2 cover the baseline and interval-estimation policies. Sections 4.3–4.6 cover the five Knowledge-Gradient variants.

4.1 Random baseline

π_{rand} draws $\theta_{n+1} \sim \text{Uniform}[\theta_{\min}, \theta_{\max}]$ each step. The belief model is not consulted. Provides a control against which informed policies must justify their added complexity.

4.2 Interval Estimation (IE / LCB)

The interval-estimation (equivalently, lower-confidence-bound) score is

$$\text{IE}(x) = \mu_n(x) - z_\alpha \cdot \sigma_n(x). \quad (3)$$

The policy picks $\theta_{n+1} = \arg \min_{x \in \mathcal{G}} \text{IE}(x)$. The parameter $z_\alpha \geq 0$ controls the exploration–exploitation tradeoff:

- $z_\alpha = 0$: pure exploitation ($\arg \min \mu_n$).
- $z_\alpha \approx 1.96$: 95% lower confidence bound.
- $z_\alpha \rightarrow \infty$: pure exploration ($\arg \max \sigma_n$, i.e. maximum-entropy reduction).

The IE batch benchmark evaluates 21 values of $z_\alpha \in \{0, 0.2, 0.4, \dots, 4.0\}$.

4.3 Knowledge Gradient: offline correlated, analytic

The Knowledge Gradient at a candidate x measures the expected reduction in the minimum posterior mean after one additional observation at x :

$$\text{KG}(x) = \min_j \mu_n(x_j) - \mathbb{E}_{Z \sim \mathcal{N}(0,1)} \left[\min_j (\mu_n(x_j) + w_j(x) Z) \right], \quad (4)$$

Algorithm 1 Closed-form $\mathbb{E}_Z[\max_j(a_j + b_j Z)]$ via Frazier–Powell–Dayanik (2009).

Require: Arrays $\{a_j, b_j\}_{j=1}^m$

```

1: Sort by  $b$  ascending; break ties by keeping the largest  $a$ .
2: Deduplicate: if two consecutive entries have identical  $b$ , drop the smaller- $a$  one.
3: Initialize a monotone stack  $S \leftarrow \{(a_1, b_1, -\infty)\}$  (the third field is the left boundary).
4: for  $i = 2, \dots, m$  do
5:   loop
6:     Let  $(A, B, C)$  be the top of  $S$ .
7:      $z \leftarrow (A - a_i)/(b_i - B)$ 
8:     if  $z \leq C$  then  $\triangleright$  top is dominated by line  $i$  on its active region
9:       Pop  $S$ ; if empty, push  $(a_i, b_i, -\infty)$ ; break.
10:    else
11:      Push  $(a_i, b_i, z)$  onto  $S$ ; break.
12:    end if
13:  end loop
14: end for
15: Let  $S = \{(A_k, B_k, C_k)\}_{k=1}^K$  with  $C_{K+1} = +\infty$ .
16: return  $\sum_{k=1}^K [A_k(\Phi(C_{k+1}) - \Phi(C_k)) + B_k(\phi(C_k) - \phi(C_{k+1}))]$ 

```

where the minimum is taken over the search grid $\{x_j\}_{j=1}^m$, and $w_j(x)$ is the *influence vector* of an observation at x on the posterior mean at x_j :

$$w_j(x) = \frac{\text{Cov}_n(x, x_j)}{\sqrt{\text{Var}_n(x) + \tilde{\sigma}^2}}. \quad (5)$$

This uses the full posterior covariance $\text{Cov}_n(x, x_j)$ — hence *correlated beliefs*. An observation at x shifts the posterior mean at *every* grid point j in proportion to how correlated x_j is with x under the GP.

Closed-form expectation. Under the KG-CB formulation of Frazier, Powell & Dayanik [2], the expectation in Eq. (4) admits a closed form based on the upper envelope of m affine functions of a standard-normal variable. The package computes $\mathbb{E}_Z[\max_j(a_j + b_j Z)]$ using Algorithm 1. To handle the minimum in Eq. (4), we use the identity $\min_j(\mu_j + w_j Z) = -\max_j(-\mu_j - w_j Z)$, so

$$\mathbb{E}_Z \left[\min_j (\mu_n(x_j) + w_j(x) Z) \right] = -\mathbb{E}_Z \left[\max_j (-\mu_n(x_j) + (-w_j(x)) Z) \right].$$

The two per-segment integrals use

$$\int_a^b \phi(z) dz = \Phi(b) - \Phi(a), \quad \int_a^b z \phi(z) dz = \phi(a) - \phi(b),$$

where ϕ is the standard-normal pdf and Φ its cdf.

Complexity. Sorting is $O(m \log m)$; the envelope construction is amortized $O(m)$; the integral sum is $O(K) \leq O(m)$. Total per candidate: $O(m \log m)$. Evaluating KG at all m candidates: $O(m^2 \log m)$ — trivial at $m = 100$.

Determinism. Algorithm 1 is exact (up to floating-point round-off), so KG evaluated this way is completely deterministic given the observations. The `rng` argument to `propose()` is accepted for interface compatibility but unused.

4.4 Knowledge Gradient: offline correlated, Monte Carlo

The same estimand as Section 4.3, but the expectation in Eq. (4) is estimated by naive Monte Carlo:

$$\widehat{\text{KG}}_{\text{MC}}(x) = \min_j \mu_n(x_j) - \frac{1}{n_{\text{mc}}} \sum_{k=1}^{n_{\text{mc}}} \min_j (\mu_n(x_j) + w_j(x) Z^{(k)}), \quad Z^{(k)} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1).$$

Default $n_{\text{mc}} = 50$. This variant exists purely for pedagogical comparison with the analytic version: with enough samples the two converge, but the MC estimator has variance $O(1/n_{\text{mc}})$ that is visible at moderate sample counts.

4.5 Knowledge Gradient: offline independent

The *independent-beliefs* KG replaces the influence vector Eq. (5) with the assumption that an observation at x shifts only the belief at x itself — no covariance cascade to other grid points. Concretely, for a candidate x let

$$M = \min_{j \in \mathcal{G}} \mu_n(x_j), \quad \delta(x) = \frac{\text{Var}_n(x)}{\sqrt{\text{Var}_n(x) + \tilde{\sigma}^2}},$$

so that the future posterior mean at x (over the random observation Y) is distributed as $\mu_n(x) + \delta(x)Z$, $Z \sim \mathcal{N}(0, 1)$. The future minimum over the extended search set $\mathcal{G} \cup \{x\}$ is $\min(M, \mu_n(x) + \delta(x)Z)$.

Let $a = M - \mu_n(x)$ and $t = a/\delta(x)$. Then

$$\mathbb{E}_Z[\min(M, \mu_n(x) + \delta(x)Z)] = M - a \Phi(t) - \delta(x) \phi(t),$$

and, letting $\text{cur_min} = \min(M, \mu_n(x))$, the independent-KG score is

$$\text{KG}_{\text{indep}}(x) = \text{cur_min} - (M - a \Phi(t) - \delta(x) \phi(t)). \quad (6)$$

This is a closed form; no numerical integration or Monte Carlo required.

Note on the two independence assumptions. There are two places in a learning loop where one might impose independence: (a) when *computing* the acquisition score, and (b) when *updating*

the belief after observing. This package always uses the full-correlation GP update in (b); the independent flavor only affects (a).

4.6 Online Knowledge Gradient (Ryzhov)

The online KG (see Ryzhov [3], Powell & Ryzhov [4]) is designed for problems where each measurement’s realized cost counts against the objective — i.e. cumulative cost matters, not only the final decision. For a minimization problem with N measurements remaining, define

$$\text{OKG}(x) = \mu_n(x) - (N - n) \cdot \text{KG}(x), \quad (7)$$

where n is the number of measurements already taken and $\text{KG}(x)$ is any of the offline KG variants above. The online policy picks $\theta_{n+1} = \arg \min_{x \in \mathcal{G}} \text{OKG}(x)$.

Two intuitions:

- Early in the run ($n \ll N$), the info-value term $(N - n)\text{KG}(x)$ dominates and the policy behaves like offline KG — exploration is rewarded because there are many future decisions to benefit from the information.
- Late in the run ($n \rightarrow N$), the multiplier shrinks to zero and the policy reduces to *pure exploitation* of the current posterior mean ($\arg \min \mu_n$).

The package implements OKG in two flavors: `OKGCorrelatedPolicy` uses the analytic correlated KG, and `OKGIndependentPolicy` uses the independent KG.

4.7 Summary of KG variants

Class name	Off/Online	Belief model	Estimator
<code>KGPoly</code>	Offline	Correlated	Analytic (FPD 2009)
<code>KGMCPoly</code>	Offline	Correlated	Monte Carlo (n_{mc} draws)
<code>KGIndependentPolicy</code>	Offline	Independent	Analytic (closed form)
<code>OKGCorrelatedPolicy</code>	Online	Correlated	Analytic KG plus μ_n tradeoff
<code>OKGIndependentPolicy</code>	Online	Independent	Analytic KG plus μ_n tradeoff

Table 1: The five KG variants in the package.

5 Session workflow

A *session* coordinates a single learning run. The learner is initialized with a belief model, an acquisition policy, and a session seed. At each step:

1. The policy proposes $\theta_{n+1} = \pi(\text{belief}, \text{rng})$.
2. The simulator is called: $y_{n+1} = \text{simulate}(\text{cfg}, \theta_{n+1}, H, \text{seed}, n)$.
3. The belief is updated: $(\mu_n, \sigma_n) \rightarrow (\mu_n[n + 1], \sigma_n[n + 1])$ given the new $(x, y) = (\theta_{n+1}, y_{n+1})$ pair.

4. The step counter increments $n \rightarrow n + 1$.

The Common Random Numbers (CRN) contract of Section 2.4 is preserved by passing `experiment_index=n` on step n . This means that different *policies* in the same session slot (same seed, same step index) see the same random draws in the simulator — differences in observed cost reflect only differences in the chosen θ , not different noise. This is critical for the fairness of batch benchmarking (Section 6).

Current-best pick. The session’s current best guess of the optimum θ is

$$\hat{\theta}_n \equiv \arg \min_{x \in \mathcal{G}_{\text{fine}}} \mu_n(x),$$

where $\mathcal{G}_{\text{fine}}$ is a fine grid (default 200 points) that need not coincide with the acquisition grid $\mathcal{G}$.

6 Batch benchmarking

The batch benchmarking framework runs an entire *family* of policies, each S times, and reports per-policy aggregated performance metrics.

6.1 Families

- **KG family (5 policies):** the five variants listed in Section 4.7.
- **IE family (21 policies):** IEPolicy at $z_\alpha \in \{0, 0.2, 0.4, \dots, 4.0\}$.

6.2 Common Random Numbers across policies

For each simulation index $s \in \{1, \dots, S\}$ a base seed is chosen as `seeds = session_seed + 10,000 s`. *Every* policy in the family runs with `seeds` at simulation index s . This means that if two policies happen to pick the same θ at the same step index, they see identical noise; differences in observed cost reflect only policy differences.

6.3 Metrics

For each individual run (one policy, one simulation index) the package records three raw quantities.

Terminal $\hat{\theta}$. The session’s current-best pick after all N measurements, $\hat{\theta}_N$.

Terminal cost. An unbiased estimate of $F(\hat{\theta}_N)$ via ground-truth Monte Carlo:

$$\text{TermCost} = \frac{1}{n_{\text{rep}}} \sum_{k=1}^{n_{\text{rep}}} \text{simulate}(\text{cfg}, \hat{\theta}_N, H, \text{seed}_s + 999,000, k). \text{total_cost}.$$

Default $n_{\text{rep}} = 12$. This measures the quality of the *final decision* — how much you would pay to run the fund at your final estimate of the optimum.

Cumulative cost. The sum of all realized costs incurred during the run:

$$\text{CumCost} = \sum_{n=1}^N y_n.$$

This measures the *cost of learning* — how expensive the experimentation was.

6.4 Ground truth reference

The true minimum of F is estimated once per batch by evaluating a 30-point grid over $[\theta_{\min}, \theta_{\max}]$ with $n_{\text{rep}} = 12$ Monte-Carlo replications at each point:

$$\hat{F}_{\text{true}}(\theta_j) = \frac{1}{n_{\text{rep}}} \sum_{k=1}^{n_{\text{rep}}} \text{simulate}(\cdot, \theta_j, \cdot, s^*, k). \text{total_cost},$$

with $s^* = \text{session_seed} + 777,000$. The reported `true_best_c_star` is $\arg \min_j \hat{F}_{\text{true}}(\theta_j)$ and `true_min_cost` is the corresponding minimum. The cumulative-cost chart uses the reference $N \cdot \text{true_min_cost}$: the theoretical floor if the learner had known the optimum from step 1 and played it every step.

6.5 Aggregation

For each policy π , the batch reports:

$$\begin{aligned} \bar{c}_\pi &= \frac{1}{S} \sum_{s=1}^S \hat{\theta}_N^{(\pi,s)}, & \sigma_{c,\pi} &= \text{std}\{\hat{\theta}_N^{(\pi,s)}\}, \\ \bar{T}_\pi &= \frac{1}{S} \sum_{s=1}^S \text{TermCost}^{(\pi,s)}, & \sigma_{T,\pi} &= \text{std}\{\text{TermCost}^{(\pi,s)}\}, \\ \bar{K}_\pi &= \frac{1}{S} \sum_{s=1}^S \text{CumCost}^{(\pi,s)}, & \sigma_{K,\pi} &= \text{std}\{\text{CumCost}^{(\pi,s)}\}. \end{aligned}$$

Both charts in the batch results view display the means with $\pm 1\sigma$ error bars.

7 Configuration parameters

Table 2 lists the simulation-model parameters, Table 3 lists the belief-model parameters, and Table 4 lists the acquisition-policy parameters. Values in the “Default” column are the ones shipped in `SimConfig`, `BeliefConfig`, and `AcquisitionConfig` respectively.

8 Software architecture

The package is organized into four Python modules and a React frontend.

sim/

Pure-function simulator. Given a config, θ , horizon, and seed pair, returns a deterministic `SimResult` (Section 2). No dependency on the learning modules.

Symbol / field	Default	Description
A_0 (<code>initial_aum</code>)	10^6	Fund initial assets under management (\$).
μ_{net} (<code>mu_net_annual</code>)	0	Annual drift of retail net flow / AUM.
σ_{net} (<code>sigma_net_annual</code>)	0.02	Annual volatility of retail flow / AUM.
λ (<code>jump_rate_annual</code>)	12	Poisson rate (jumps/year).
μ_{jump} (<code>jump_mean_log</code>)	-4.5	Mean of $\log(J /\text{AUM})$; median jump $\approx 1.1\%$.
σ_{jump} (<code>jump_std_log</code>)	0.5	Std of $\log(J /\text{AUM})$.
p_{in} (<code>jump_inflow_prob</code>)	0.5	Probability jump is an inflow.
<code>stationary</code>	<code>False</code>	If <code>True</code> , disable regime switching.
$(\mu_0^{\text{reg}}, \mu_1^{\text{reg}})$	(0.001, -0.001)	Regime drift adjustments (annual).
p_{switch}	0.01	Daily regime-switch probability.
r_{mkt} (<code>r_market_annual</code>)	0.07	Market portfolio return (annual).
r_{cash} (<code>r_cash_annual</code>)	0.04	Cash return (annual).
r_{borrow} (<code>r_borrow_annual</code>)	0.10	Shortfall penalty rate (per \$ of forced liquidation).
<code>opp_cost_on_total_cash</code>	<code>True</code>	Apply drag to all cash (<code>True</code>) or only excess (<code>False</code>).
ρ (<code>rebalance_speed</code>)	1.0	Fraction of gap closed each end-of-day rebalance.
$[\theta_{\min}, \theta_{\max}]$	[0.01, 0.20]	Legal range of θ .
<code>trading_days_per_year</code>	252	Calendar convention.

Table 2: Simulation model parameters (`SimConfig`).

Symbol / field	Default	Description
ℓ (<code>length_scale</code>)	0.04	RBF kernel length scale (in θ units).
σ_f (<code>signal_std</code>)	5000	Prior amplitude of $F(\theta)$ (cost units).
$\tilde{\sigma}$ (<code>noise_std</code>)	3000	Assumed per-observation noise std.
m_0 (<code>prior_mean</code>)	5000	Prior mean of F .
<code>jitter</code>	10^{-6}	Diagonal jitter for Cholesky stability.

Table 3: Belief model parameters (`BeliefConfig`).

`policy/`

Belief model (`belief.py`), acquisition policies (`acquire.py`), and session coordinator (`session.py`).

All acquisition policies satisfy the protocol `propose(model, rng) -> float`.

`shell/`

FastAPI service exposing `POST /sessions`, `POST /sessions/{sid}/step`, `GET /sessions/{sid}/posterior`, `GET /sessions/{sid}/reveal`, `GET /sessions/{sid}/kg`, and `POST /sessions/batch`. The batch endpoint streams NDJSON progress events.

`frontend/`

React + Vite single-page app. Two primary views: (a) live session (posterior chart, KG comparison chart, aligned θ slider for human mode), (b) batch benchmark results (mean-terminal-cost and mean-cumulative-cost bar charts with $\pm 1\sigma$ error bars, plus aggregate table).

A test suite of ≈ 145 unit tests exercises every acquisition policy, the GP posterior formulas, the CRN contract, and the batch endpoint.

Symbol / field	Default	Description
$\theta_{\min}, \theta_{\max}$	0.01, 0.20	Acquisition-grid endpoints (mirror <code>SimConfig</code>).
M (<code>grid_size</code>)	100	Number of grid points $\mathcal{G}$.
z_{α} (<code>z_alpha</code>)	0.0	Exploration weight in IE (see Eq. (3)).

Table 4: Acquisition config parameters (`AcquisitionConfig`).

References

- [1] P. I. Frazier, W. B. Powell, and S. Dayanik, “A knowledge-gradient policy for sequential information collection,” *SIAM Journal on Control and Optimization*, vol. 47, no. 5, pp. 2410–2439, 2008.
- [2] P. I. Frazier, W. B. Powell, and S. Dayanik, “The knowledge-gradient policy for correlated normal beliefs,” *INFORMS Journal on Computing*, vol. 21, no. 4, pp. 599–613, 2009.
- [3] I. O. Ryzhov, W. B. Powell, and P. I. Frazier, “The knowledge gradient algorithm for a general class of online learning problems,” *Operations Research*, 2010.
- [4] W. B. Powell and I. O. Ryzhov, *Optimal Learning*, Wiley, 2012.
- [5] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006.